**MINISTRY OF EDUCATION AND RESEARCH**
**OF THE REPUBLIC OF MOLDOVA**
**Technical University of Moldova**
**Faculty of Computers, Informatics and Microelectronics**
**Department of Software Engineering and Automation**

**Daniel Moisei, FAF-241**

# Report

*Laboratory Work No. 3*

***Cryptography and Security***

Topic: Playfair Cipher

Digraph Substitution for the Romanian Alphabet

Checked by:
**Maia Zaica**, *university assistant*

**Chişinău – 2026**

# CONTENTS

**Introduction** **3**

**1 Short Theory** **4**

1.1 Playfair Cipher . . . . . 4

1.2 Plaintext Preparation . . . . . 4

1.3 Encryption Rules . . . . . 4

**2 Implementation** **5**

2.1 Program Structure . . . . . 5

2.2 Interactive Execution . . . . . 5

2.3 Matrix Construction . . . . . 6

2.4 Preparing Letter Pairs . . . . . 7

2.5 Pair Transformation . . . . . 8

**3 Results** **9**

3.1 Encryption and Decryption with Romanian Letters . . . . . 9

3.2 Repeated-Letter Preparation . . . . . 9

3.3 Invalid Input and Key Preservation . . . . . 10

**4 Conclusions** **11**

**5 Source Code** **11**

# INTRODUCTION

The purpose of this laboratory work is to study and implement the Playfair cipher, a classical digraph substitution method. According to the assignment rule, students with an odd number in the group register complete Task 3.1. Since the assigned register number is 17, this report presents the Playfair implementation rather than the Vigenère task.

The application was implemented in C# as an interactive console program for the 31-letter Romanian alphabet. It supports both encryption and decryption, constructs a key-dependent matrix, prepares plaintext as letter pairs, validates the key and message, and continues running until the user explicitly selects the exit option. Invalid message or cryptogram input is requested again without discarding the already accepted key and matrix.

# 1 SHORT THEORY

## 1.1 Playfair Cipher

The Playfair cipher encrypts pairs of letters instead of processing every letter independently. This makes it a polygraphic substitution cipher: the result assigned to one letter depends on the other letter in the same pair. Compared with a simple monoalphabetic substitution, single-letter frequency information is less directly visible because the algorithm operates on digraphs.

The cipher uses a matrix produced from a secret keyword. Repeated letters are removed from the keyword, the unused alphabet letters are appended, and the resulting sequence is written into the matrix row by row. The same matrix is required for both encryption and decryption.

The Romanian alphabet used by the program is

AĂÂBCDEFGHIÎJKLMNOPQRSŞTŢUVWXYZ.

It contains 31 letters. Since 31 is a prime number, these symbols cannot fill a nontrivial rectangular matrix exactly. The implementation therefore uses a $4 \times 8$ matrix and fills the remaining cell with #. This follows the laboratory description, which permits unused matrix cells to contain an additional symbol. The extra symbol is accepted only when a cryptogram is entered for decryption.

## 1.2 Plaintext Preparation

Before encryption, spaces are removed and letters are converted to uppercase. The normalized plaintext is divided into pairs. If the two letters of a pair are equal, a filler letter is inserted between them. If one letter remains at the end, another filler is appended. The program normally uses X; when the original letter is already X, it uses Q to avoid creating the pair XX.

For example,

$$\texttt{BALLOON} \longrightarrow \texttt{BA LX LO ON}. \tag{1}$$

The inserted filler letters may be ambiguous in a naturally occurring message. Therefore, as required by the assignment, filler removal and the restoration of word spacing are performed manually after decryption.

## 1.3 Encryption Rules

Let the current pair be $(a, b)$. Encryption applies one of three rules:

1. If $a$ and $b$ are on the same row, replace each with the symbol immediately to its right. The last column wraps to the first column.

2. If $a$ and $b$ are in the same column, replace each with the symbol immediately below it. The last row wraps to the first row.

3. Otherwise, the letters form opposite corners of a rectangle. Replace each with the symbol on its own row and the other letter's column.

For decryption, the rectangle rule is unchanged. The same-row rule moves one position to the left, and the same-column rule moves one position upward.

# 2 IMPLEMENTATION

## 2.1 Program Structure

The console entry point is intentionally small. It prints the laboratory title and delegates the complete workflow to the public `Playfair.Run()` method.

Listing 1: Console entry point in Program.cs

```
namespace Lab03
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine(
                "Lab 03 - Playfair Cipher for the Romanian Alphabet"
            );
            Playfair.Run();
        }
    }
}
```

The implementation constants define the Romanian alphabet and the dimensions of the matrix. A dictionary maps every symbol to its row and column so that pair transformation does not require scanning the complete matrix repeatedly.

Listing 2: Alphabet, matrix, and position lookup

```
private const string Alphabet =
    "AĂÂBCDEFGHIÎJKLMNOPQRSŞTŢUVWXYZ";
private const char EmptyCell = '#';
private const int Rows = 4;
private const int Columns = 8;

private static readonly char[,] Matrix =
    new char[Rows, Columns];

private static readonly Dictionary<
    char, (int Row, int Column)> Positions = new();
```

## 2.2 Interactive Execution

The `Run` method keeps displaying the operation menu until the user enters `X` or `EXIT`. Each selected operation obtains and validates a key, constructs the corresponding matrix, and then calls the encryption or decryption workflow.

Listing 3: Continuous operation loop

```
public static void Run()
{
    Console.OutputEncoding = Encoding.UTF8;
    Console.InputEncoding = Encoding.UTF8;
    Console.WriteLine($"Alfabet: {Alphabet}");

    while (true)
    {
        Console.WriteLine();
        string operation = ReadOperation();

        if (operation is "X" or "EXIT")
        {
            Console.WriteLine("La revedere!");
            return;
        }

        string key = ReadKey();
        BuildMatrix(key);

        Console.WriteLine("\nMatricea Playfair:");
        PrintMatrix();

        if (operation == "E")
            EncryptMessage();
        else
            DecryptMessage();
    }
}
```

Input validation is performed at the field where the error occurs. Consequently, an invalid message or cryptogram does not force the user to select the operation or enter the key again. The existing matrix remains active until a valid value is supplied.

### 2.3 Matrix Construction

Matrix construction concatenates the key, alphabet, and extra-cell symbol. The duplicate-removal method retains only the first occurrence of each symbol. The resulting 32 characters are placed in a $4 \times 8$ array, while the dictionary records their coordinates.

Listing 4: Construction of the keyed Playfair matrix

```
private static void BuildMatrix(string key)
{
    string symbols = RemoveDuplicates(
        key + Alphabet + EmptyCell
    );
    Positions.Clear();
```

```

    for (int index = 0;
         index < Rows * Columns;
         index++)
    {
        int row = index / Columns;
        int column = index % Columns;
        char symbol = symbols[index];

        Matrix[row, column] = symbol;
        Positions[symbol] = (row, column);
    }
}
```

For the keyword `SECURITATE`, the generated matrix is

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| S | E | C | U | R | I | T | A |
| Ă | Â | B | D | F | G | H | Î |
| J | K | L | M | N | O | P | Q |
| Ş | Ţ | V | W | X | Y | Z | # |

### 2.4 Preparing Letter Pairs

The plaintext-preparation loop consumes either one or two source letters. It consumes one letter when it inserts a filler between equal adjacent letters; otherwise, it consumes the complete pair. This prevents a repeated plaintext letter from being lost.

Listing 5: Plaintext division into valid digraphs

```
for (int index = 0; index < normalized.Length;)
{
    char first = normalized[index];
    char second;

    if (index + 1 >= normalized.Length)
    {
        second = ChooseFiller(first);
        index++;
    }
    else if (first == normalized[index + 1])
    {
        second = ChooseFiller(first);
        index++;
    }
    else
    {
        second = normalized[index + 1];
        index += 2;
```

```
    }

    result.Append(first);
    result.Append(second);
}
```

### 2.5 Pair Transformation

Encryption and decryption share the same transformation method. The direction is $+1$ for encryption and $-1$ for decryption. The helper `Mod` provides positive wraparound when moving backward from the first row or column.

Listing 6: Same-row, same-column, and rectangle rules

```
int direction = encrypt ? 1 : -1;

if (firstRow == secondRow)
{
    result.Append(Matrix[
        firstRow,
        Mod(firstColumn + direction, Columns)]);
    result.Append(Matrix[
        secondRow,
        Mod(secondColumn + direction, Columns)]);
}
else if (firstColumn == secondColumn)
{
    result.Append(Matrix[
        Mod(firstRow + direction, Rows),
        firstColumn]);
    result.Append(Matrix[
        Mod(secondRow + direction, Rows),
        secondColumn]);
}
else
{
    result.Append(Matrix[firstRow, secondColumn]);
    result.Append(Matrix[secondRow, firstColumn]);
}
```

# 3 RESULTS

The project was compiled with the .NET SDK before testing. The build completed with zero warnings and zero errors. Functional checks covered encryption, decryption, Romanian diacritics, repeated plaintext letters, invalid characters, odd-length cryptograms, persistent validation, and explicit exit.

## 3.1 Encryption and Decryption with Romanian Letters

The keyword SECURITATE was used to verify all three transformation rules and Romanian diacritics. Spaces were removed automatically from the tested message, with the following result:

MÂINE ÎNVĂȚĂM ⟶ KDROAÂLXÂŞDJ.

Figure 1: Encryption test

```
Alegeți operația: E
Introduceți cheia (minimum 7 litere): SECURITATE

Matricea Playfair:
S E C U R I T A
Ă Â B D F G H Î
J K L M N O P Q
Ș Ț V W X Y Z #

Introduceți mesajul: MÂINE ÎNVĂȚĂM
Mesaj pregătit: MÂINEÎNVĂȚĂM
Criptograma: KDROAÂLXÂȘDJ
```

Entering the generated cryptogram with the same key recovered the normalized plaintext exactly.

Figure 2: Decryption test

```
Alegeți operația: D
Introduceți cheia (minimum 7 litere): SECURITATE
Introduceți criptograma: KDROAÂLXÂȘDJ
Mesaj decriptat: MÂINEÎNVĂȚĂM
Eliminați manual literele de umplere și adăugați spațiile.
```

## 3.2 Repeated-Letter Preparation

The message BALLOON contains two consecutive L letters. The application inserted X between them, producing the valid digraph sequence BA LX LO ON. The prepared message and ciphertext were

Figure 3: Insertion of a filler between repeated letters

```
Introduceți mesajul: BALLOON
Mesaj pregătit: BALXLOON
Criptograma: ÎCNVMPPO
```

### 3.3 Invalid Input and Key Preservation

An unsupported symbol and an odd-length cryptogram were entered before the valid cryptogram. In both cases, the program reported the exact problem and requested only the cryptogram again. The keyword and generated matrix were preserved.

Figure 4: Field-level validation without re-entering the key

```
Introduceți criptograma: ABC!
Eroare: Caracterul '!' nu este permis.
Încercați din nou; cheia și matricea au fost păstrate.

Introduceți criptograma: ABC
Eroare: Criptograma trebuie să conțină un număr par
de caractere.
Încercați din nou; cheia și matricea au fost păstrate.

Introduceți criptograma: KDROAÂLXÂȘDJ
Mesaj decriptat: MÂINEÎNVĂȚĂM
```

Table 1: Summary of verified behavior

| **Test** | **Observed result** |
|---|---|
| Build | Successful; zero warnings and zero errors |
| Romanian-letter encryption | Produced the expected 12-character cryptogram |
| Decryption round trip | Recovered the normalized plaintext exactly |
| Repeated letters | Inserted `X` without losing either original letter |
| Invalid character | Reported the rejected symbol and requested the field again |
| Odd cryptogram length | Rejected before pair transformation |
| Persistent menu | Continued until the explicit exit command |

## 4 CONCLUSIONS

This laboratory work demonstrated how the Playfair cipher extends substitution from individual letters to digraphs. The central implementation tasks were constructing a deterministic matrix from the key, preparing valid letter pairs, locating symbols efficiently, and applying inverse movement rules for decryption. A coordinate dictionary made the pair transformation direct and kept the three Playfair cases easy to distinguish in the source code.

Adapting the method to Romanian required an explicit 31-letter alphabet. Because 31 symbols cannot fill a nontrivial rectangular matrix, a single additional matrix symbol was used in a $4 \times 8$ layout. The program validates ordinary messages against the Romanian alphabet while accepting that extra symbol only in a cryptogram that may legitimately contain it.

The final application satisfies the functional requirements: the key contains at least seven letters, spaces are removed, letters are converted to uppercase, encryption and decryption are selectable, invalid input is explained, and word spacing is restored manually after decryption. The continuous menu and field-level retry loops improve usability without changing the cipher or contradicting the assignment.

As a historical cipher, Playfair is not appropriate for modern data protection. It hides single-letter frequencies better than a monoalphabetic substitution but still preserves patterns in digraphs and is vulnerable to modern automated cryptanalysis. Its value is educational: it provides a clear introduction to block-oriented substitution and key-dependent transformations.

## 5 SOURCE CODE

The complete C# implementation is available in the course repository:

`https://github.com/Moisei-D/CS-Labs/tree/main/labs/Lab03`

The main implementation is located in `Playfair.cs`. The public `Run()` method manages the user interaction, while private methods implement validation, matrix construction, plaintext preparation, encryption, and decryption. `Program.cs` contains only the console entry point.